NAME : K.Rishiwan
ROLL : 2420030762

VM/Container Observation of OS Isolation	Students run the same service on host and inside VM/container lab environment and compare process visibility, PID namespace differences (if applicable), resource limits, and performance overhead. They document which OS abstractions remain same and what changes.

1. Aim

To study **OS isolation** by running the same program on the **host system and inside a VM/Container**, and observe differences in **process ID (PID), hostname, system details, and resource visibility**.

2. Problem Statement

Students need to understand how **Virtual Machines (VMs) and Containers isolate operating system resources**. By running the same C program on both the **host machine and inside a container/VM**, they will compare:

- Process ID (PID)
- Parent Process ID (PPID)
- Hostname
- System information
- Running processes

3. Procedure

1. Open a Linux system or cloud instance.
2. Write the given **C program** in a file named isolation.c.
3. Compile the program using the GCC compiler.

gcc isolation.c -o isolation

Run the program on the **host machine**:

./isolation

Run the same program **inside a container or VM**.

Compare the outputs such as **PID, hostname, and process list**.

Note the differences to understand **OS isolation**.

4. PROGRAM

```c
#include <stdio.h>
#include <unistd.h>

int main()
{
    int pid, ppid;

    printf("===== OS Isolation Observation =====\n\n");

    /* Get Process ID */
    pid = getpid();

    /* Get Parent Process ID */
    ppid = getppid();

    printf("Process Information\n");
    printf("-------------------\n");
    printf("Process ID (PID)        : %d\n", pid);
    printf("Parent Process ID (PPID): %d\n", ppid);

    printf("\nObservation:\n");
    printf("If this program runs inside a container or VM,\n");
    printf("the PID and PPID may be different from the host system.\n");

    printf("\nConclusion:\n");
    printf("VMs and Containers provide OS isolation for processes.\n");

    return 0;
}
```

5. OUTPUT:

STDIN

Input for the program (Optional)

Output: 2 ms | 4.5 MB

```
===== OS Isolation Observation =====

Process Information
-------------------
Process ID (PID)        : 3
Parent Process ID (PPID): 2

Observation:
If this program runs inside a container or VM,
the PID and PPID may be different from the host system.

Conclusion:
VMs and Containers provide OS isolation for processes.
```

Cloud-Ready OS Service Load Test	Students load test the final service node (from MP12) under multiple clients. They measure throughput and tail latency, demonstrate graceful degradation under overload, and show resource limits preventing system collapse. They submit a small runbook describing how to deploy and operate it safely.

1. Aim

To perform **load testing on a service node** by simulating multiple client requests and measuring **throughput and response behavior**, ensuring the system handles overload conditions safely without crashing.

2. Problem Statement

Cloud-based services must handle **multiple client requests simultaneously**. If too many clients access the service, the system may slow down or fail.
This experiment demonstrates how to **simulate multiple clients sending requests to a service**, measure the **number of handled requests**, and observe **system behavior under heavy load**.

3. Procedure

1. Open a C programming environment or Linux terminal.
2. Create a file named **loadtest.c**.
3. Write the given C program.
4. Compile the program using the GCC compiler.
5. Run the program to simulate multiple clients.
6. Observe the number of processed requests and system response.

4. C Program

```c
#include <stdio.h>
#include <pthread.h>
#include <unistd.h>

#define CLIENTS 5

void* client_request(void* arg)
{
    int id = *(int*)arg;

    printf("Client %d sending request to server...\n", id);

    // Simulate server processing time
    sleep(2);

    printf("Server processed request from Client %d\n", id);

    pthread_exit(NULL);
}

int main()
{
    pthread_t threads[CLIENTS];
    int client_id[CLIENTS];
    int i;

    printf("=== Cloud Service Load Test Simulation ===\n\n");

    // Create multiple client requests
    for(i = 0; i < CLIENTS; i++)
    {
        client_id[i] = i + 1;
        pthread_create(&threads[i], NULL, client_request, &client_id[i]);
    }

    // Wait for all clients to finish
    for(i = 0; i < CLIENTS; i++)
    {
        pthread_join(threads[i], NULL);
    }

    printf("\nAll client requests processed successfully.\n");
    printf("System handled multiple requests without failure.\n");

    return 0;
}
```

5.OUTPUT:

STDIN

Input for the program (Optional)

Output:

```
=== Cloud Service Load Test Simulation ===

Client 1 sending request to server...
Client 3 sending request to server...
Client 4 sending request to server...
Client 5 sending request to server...
Client 2 sending request to server...
Server processed request from Client 1
Server processed request from Client 3
Server processed request from Client 4
Server processed request from Client 5
Server processed request from Client 2

All client requests processed successfully.
System handled multiple requests without failure.
```